SMARTPICK AI

Implementation Guide

Step-by-step build plan, commands, module responsibilities, and day-by-day deliverables

Prepared for: Pratyush Mishra
Version 1.0 • 15 October 2026
Status: Working companion document

1. Purpose

- This guide converts the SmartPick AI PRD and TRD into an implementation sequence. Build a reliable baseline first, then add similarity ranking and evaluation.

2. Build order

- Day 1 — Select a dataset with documented source/licence; create data/phones.csv; validate required columns and types.
- Day 2 — Implement budget filtering and weighted ranking in src/recommender.py; write unit tests.
- Day 3 — Build Streamlit controls and recommendation cards; verify an end-to-end local demo.
- Days 4–5 — Add fitted preprocessing, content-based similarity, baseline comparison, and evaluation where relevance labels exist.
- Day 6 — Write README, requirements, data provenance notes, screenshots, and attempt deployment.
- Day 7 — Prepare slides, demo script, and viva answers; verify all claims against the implementation.

3. Suggested setup

- Create a Python 3.11 virtual environment, install pandas, numpy, scikit-learn, streamlit, plotly, joblib, and pytest. Use a tested/pinned requirements file before release.
- Run the app locally with: streamlit run app.py
- Run tests with: pytest

4. Core modules

- app.py — Streamlit user interface and rendering only.
- src/data_loader.py — Read CSV, validate columns, and report data issues.
- src/preprocessing.py — Handle missing values and prepare numeric/categorical features.
- src/recommender.py — Budget constraints, ranking, similarity, and explanations.
- src/evaluation.py — Offline metrics and baseline comparison.
- tests/test_recommender.py — Unit and edge-case tests.

5. Definition of done

- The app starts from a clean environment using documented steps.
- Budget constraints are respected for every result.
- Invalid inputs and empty results show understandable messages.
- Recommendation reasons use only real available attributes.
- Core logic has automated tests.
- Dataset provenance and licence are documented.
- Evaluation metrics are reported only when their required labels exist.
- Presentation and screenshots match the actual implementation.

6. Important cautions

- A weighted ranking formula is a baseline, not a trained model.
- Synthetic labels can test the code but cannot prove real-world recommendation quality.
- Do not invent prices, benchmark scores, ratings, or current availability.
- Do not publish raw dataset files unless its licence permits redistribution.